Parallel Bubble Sort and Merge Sort using OpenMP

1. Aim of the Practical

To implement:

- Parallel Bubble Sort
- Parallel Merge Sort

using:

- OpenMP
- Existing sorting algorithms

and compare:

- Sequential execution time
- Parallel execution time

---

2. What is Sorting?

Sorting means arranging data in:

- Ascending order
- Descending order

Example:

Before sorting:

5 2 8 1 9

After sorting:

1 2 5 8 9

---

3. What is Bubble Sort?

Bubble sort repeatedly:

- Compares adjacent elements
- Swaps them if incorrect

Largest element moves to end in every pass.

Bubble Sort Example

5 4 3 2

Pass 1:

4 3 2 5

Pass 2:

3 2 4 5

Pass 3:

2 3 4 5

---

4. Time Complexity of Bubble Sort

Worst case:

O(n^2)

Because:

- Nested loops are used.

---

5. What is Merge Sort?

Merge sort:

1. Divides array into halves
2. Sorts each half
3. Merges sorted halves

Uses:

Divide and Conquer

Merge Sort Example

8 3 5 1

Divide:

8 3 5 1

Sort:

3 8 1 5

Merge:

1 3 5 8

---

6. Time Complexity of Merge Sort

O(n\log n)

Much faster than bubble sort for large data.

---

7. What is Parallel Computing?

Parallel computing means:

- Multiple operations execute simultaneously
- Uses multiple CPU cores

Benefits:

- Faster execution
- Better performance

---

8. What is OpenMP?

OpenMP = Open Multi-Processing

Used for:

- Parallel programming in C/C++

Creates:

- Multiple threads automatically

---

9. Header Files Used

```
#include <iostream>
#include <vector>
#include <omp.h>
#include <chrono>
```

---

10. Purpose of Header Files

Header	Purpose
iostream	Input/output
vector	Dynamic arrays
omp.h	OpenMP functions
chrono	Measure execution time

---

11. Using Namespace

using namespace std;

using namespace chrono;

Simplifies code writing.

---

12. Parallel Bubble Merge Class

class Parallel_bubble_merge

Contains:

- Parallel bubble sort
- Parallel merge sort
- Merge function

---

13. Parallel Bubble Sort Function

void parallel_bubble(vector<int> &arr)

Sorts array using parallel bubble sort.

---

14. Outer Loop

for (int i = 0; i < n - 1; i++)

Controls number of passes.

---

15. Parallel Loop

#pragma omp parallel for

MOST IMPORTANT CONCEPT

Parallelizes inner loop.

Different threads compare elements simultaneously.

---

16. Inner Loop

for (int j = 0; j < n - i - 1; j++)

Compares adjacent elements.

---

17. Swapping Elements

```
if (arr[j] > arr[j + 1])
{
   swap(arr[j], arr[j + 1]);
}
```

If left element is larger:

- Swap elements

---

18. Merge Function

void merge(vector<int> &arr, int low, int mid, int high)

Combines two sorted halves into one sorted array.

Example

Before merge:

Left : 1 4 7

Right: 2 5 8

After merge:

1 2 4 5 7 8

---

19. Temporary Array

vector<int> temp;

Stores merged sorted elements temporarily.

---

20. Comparing Elements

if (arr[i] < arr[j])

Smaller element added to temp array first.

---

21. Copy Remaining Elements

while (i <= mid)

Copies leftover left-side elements.

while (j <= high)

Copies leftover right-side elements.

---

22. Copy Back to Original Array

arr[i] = temp[i - low];

Merged data copied back into original array.

---

23. Parallel Merge Sort

void parallel_merge(vector<int> &arr, int low, int high)

Recursive parallel merge sort function.

---

24. Base Condition

if (low < high)

Stops recursion when:

- Array contains one element

---

25. Finding Mid

int mid = (low + high) / 2;

Splits array into two halves.

---

26. Parallel Sections

#pragma omp parallel sections

MOST IMPORTANT CONCEPT

Executes different sections simultaneously.

---

27. OpenMP Sections

#pragma omp section

parallel_merge(arr, low, mid);

#pragma omp section

parallel_merge(arr, mid + 1, high);

Two halves sorted in parallel.

---

28. Why Merge Sort is Good for Parallelism?

Because:

- Left half independent of right half
- Both can execute simultaneously

Hence merge sort is highly parallelizable.

---

29. Sequential Bubble Merge Class

class sequential_bubble_merge

Contains:

- Sequential bubble sort
- Sequential merge sort

---

30. Sequential Bubble Sort

void sequential_bubble(vector<int> &arr)

Runs normally without threads.

---

31. Sequential Merge Sort

void sequential_merge(vector<int> &arr, int low, int high)

Recursively sorts array.

NOTE ABOUT YOUR CODE

Inside sequential merge:

#pragma omp parallel sections

should NOT be present in true sequential implementation.

For pure sequential merge sort:
Remove OpenMP directives.

---

32. Creating Array

```
for (int i = 500; i >= 1; i--)
{
    arr1.push_back(i);
```

}

Creates reverse sorted array:

500 499 498 ... 1

Worst case input.

---

33. Copying Arrays

vector<int> arr2 = arr1;

Different copies used for:

- Sequential sort
- Parallel sort

Ensures fair comparison.

---

34. Measuring Time

auto start = high_resolution_clock::now();

Starts timer.

---

35. Ending Timer

auto end = high_resolution_clock::now();

Stops timer.

---

36. Calculating Duration

duration_cast<microseconds>(end - start);

Measures execution time in:

- Microseconds

---

37. Sequential Bubble Sort Timing

s.sequential_bubble(arr1);

Runs sequential bubble sort.

---

38. Parallel Bubble Sort Timing

p.parallel_bubble(arr2);

Runs parallel bubble sort.

---

39. Printing Array

print_array_parallel(arr2);

Displays sorted array.

---

40. Commented Merge Sort Code

Your merge sort timing section is commented:

// auto start2 = high_resolution_clock::now();

To execute merge sort:
Remove comments.

---

41. Correct Merge Sort Call

Instead of:

p.parallel_merge(arr3,0,500);

Use:

p.parallel_merge(arr3,0,arr3.size()-1);

Because valid index:

0 to 499

---

42. Sequential Merge Correction

Remove OpenMP directives from sequential merge.

Correct version:

sequential_merge(arr, low, mid);

sequential_merge(arr, mid + 1, high);

---

43. Expected Result

Parallel algorithms usually:

- Faster on large datasets
- Better CPU utilization

---

44. Why Parallel Bubble Sort is Less Efficient?

Bubble sort has:

- Data dependency
- Frequent swapping

So parallelization benefit is limited.

---

45. Why Parallel Merge Sort is Better?

Merge sort naturally divides work:

- Left half
- Right half

Perfect for parallel threads.

---

46. Performance Comparison

Algorithm	Time Complexity
Bubble Sort	(O(n^2))
Merge Sort	(O(n\log n))

---

47. OpenMP Concepts Used

Directive	Purpose
parallel for	Parallel loop
parallel sections	Execute sections simultaneously
section	Independent task

---

48. Compilation Command

g++ file.cpp -fopenmp

---

49. Run Command

./a.out

---

50. Important Viva Questions

Q1. What is Bubble Sort?

Compares adjacent elements and swaps them.

Q2. What is Merge Sort?

Divide-and-conquer sorting algorithm.

Q3. Why is Merge Sort faster?

Time complexity:
[
O(n\log n)
]

Q4. What is OpenMP?

API for parallel programming in C/C++.

Q5. What is parallel for?

Executes loop iterations simultaneously.

Q6. Why merge sort is better for parallelization?

Because subarrays can be processed independently.

Q7. What is speedup?

Performance improvement due to parallelism.

Formula:

Speedup=\frac{Sequential\ Time}{Parallel\ Time}

---

51. Final Conclusion

In this practical:

- Sequential and parallel Bubble Sort were implemented
- Sequential and parallel Merge Sort were implemented
- OpenMP enabled multi-threading

- Execution times were measured using chrono library
- Parallel Merge Sort showed better scalability and performance

The practical demonstrates how parallel computing improves sorting efficiency using OpenMP.